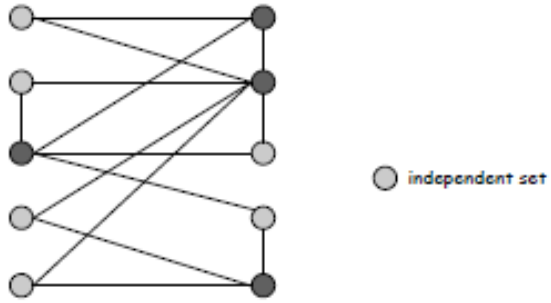
Reduceri polinomiale

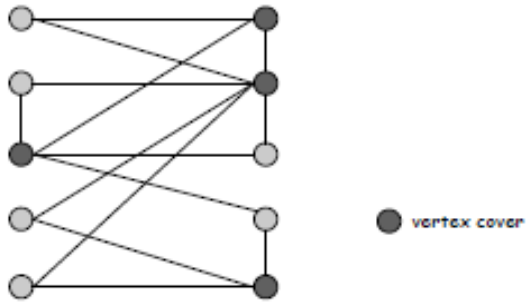
Echivalența polinomială:
Dacă $\mathbf{A} \leq_p \mathbf{B}$ și $\mathbf{B} \leq_p \mathbf{A}$, atunci $\mathbf{A} \equiv_p \mathbf{B}$

Reduceri prin simpla echivalență (Independent Set $\equiv_p$ Vertex Cover)

Independent Set: Dându-se un graf G = (V, E) și un întreg k, există un subset S de noduri a.î. $|S| \geq k$, iar fiecare muchie are **cel mult** un capăt în S? (|S| = cardinal de S)
Ex: Pt k=6 – da, există. Pt k=7 – nu, nu există.



Vertex Cover: Dându-se un graf G = (V, E) și un întreg k, există un subset S de noduri a.î. $|S| \leq k$, iar fiecare muchie are **cel puțin** un capăt în S?
Ex: Pt k=4 – da, există. Pt k=3 – nu, nu există.



[Se vede din enunț că cele două probleme sunt complementare] Arătăm că Independent Set $\equiv_p$ Vertex Cover arătând că, pt un k fixat, S este independent set de dimensiune $\geq k$ $\Leftrightarrow$ V-S este vertex cover de dimensiune $\leq |V|-k$.

Transformarea în timp polinomial a datelor de intrare:

Independent Set	**Vertex Cover**
(V,E)	(V,E)
k	|V|-k

=> (S independent set => V-S vertex cover)

- Fie S independent set și (u,v) o muchie arbitrară din G
- S independent set => $u \notin S$ sau $v \notin S$ => $u \in V-S$ sau $v \in V-S$ => V-S vertex cover

<= (V-S vertex cover => S independent set)

- Fie V-S vertex cover și (u,v) o muchie arbitrară din G
- V-S vertex cover => u∈V-S sau v∈V-S => u∉S sau v∉S => S independent set

Reduceri de la o problemă mai "ușoară" la una mai "grea" (Vertex Cover $\leq_p$ Set Cover)

Set cover: Dându-se o mulțime U, o colecție S_1, S_2 ... S_m de submulțimi ale lui U și un întreg k, există o colecție de cel mult k astfel de submulțimi care reunite să dea U?

[La Vertex Cover acoperim muchiile din E cu cât mai puține noduri, la Set Cover acoperim elementele lui U cu cât mai puține submulțimi => U va conține muchii, vom avea câte o submulțime pentru fiecare nod, iar în fiecare submulțime se vor afla muchiile pe care acel nod le atinge] Arătăm că Vertex Cover $\leq_p$ Set Cover construind dintr-o instanță oarecare a problemei Vertex Cover o instanță specifică pt Set Cover, după modelul următor:

Transformarea în timp polinomial a datelor de intrare:

Vertex Cover	**Set Cover**
(V,E), k	k = k
	U = E
	S_v = { e∈E | ∃ u∈V a.i. e=(u,v) } pt ∀v∈V

=> (S vertex cover de dimensiune ≤ k => C={ S_v | v∈S } set cover de dimensiune ≤ k)

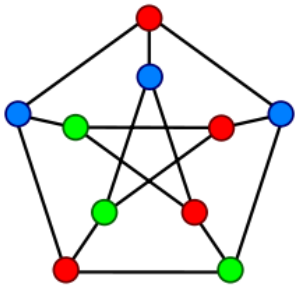
- Fie S vertex cover de dimensiune ≤ k și (u,v) o muchie arbitrară din G
- S vertex cover => u∈S sau v∈S => S_u∈C sau S_v∈C
- Din definiția lui S_i: (u,v)∈S_u, (u,v)∈S_v => C set cover

<= (C set cover de dimensiune ≤ k => S = {v | S_v∈C } vertex cover de dimensiune ≤ k)

- Fie C set cover de dimensiune ≤ k și (u,v) o muchie arbitrară din G
- C set cover => ∃ S_i∈C a.i. (u,v)∈S_i => i=u sau i=v => u∈S sau v∈S => S vertex cover

3-colorare $\leq_p$ 4-colorare

k-colorare: Dându-se un graf G = (V, E) și un întreg k, se poate colora fiecare nod din G cu una din k posibile culori a.î. să nu existe două noduri adiacente colorate la fel?
Ex: Pt k=3 – da, se poate. Pt k=2 – nu, nu se poate.



[Ideea e să obținem aceeași colorare pentru nodurile din V atât pentru instanța de tip 3-colorare cât și pentru cea de tip 4-colorare. Soluția este să adăugăm un nod z în instanța pentru 4-colorare care, în mod obligatoriu, să fie colorat cu a 4-a culoare (de ex. galben), și niciun alt nod să nu mai poată fi galben; Cum ne asigurăm de asta? Legând nodul z de fiecare nod din V] Arătăm că 3-colorare $\leq_p$ 4-colorare construind dintr-o instanță oarecare a problemei 3-colorare o instanță specifică pt 4-colorare, după modelul următor:

Transformarea în timp polinomial a datelor de intrare:

3-colorare	**4-colorare**
(V,E), k	k = k
	V = V ∪ {z}
	E = E ∪ { (v,z) | v∈V }

=> (C colorare pt 3-colorare => C ∪ { (z, galben) } colorare pt 4-colorare)

- C colorare pt 3-colorare => nodurile din V sunt colorate cu 3 culori (ex: roșu, verde, albastru), și ∄ două noduri adiacente în V colorate la fel
- V = V ∪ {z} => ∄ două noduri adiacente în V colorate la fel în C ∪ { (z, galben) }

<= (C colorare pt 4-colorare => C \ { (z, Culoare) } colorare pentru 3-colorare)

- C colorare pt 4-colorare => nodurile din V sunt colorate cu 4 culori (ex: roșu, verde, albastru, galben), și ∄ două noduri adiacente în V colorate la fel
- V = V ∪ {z} și E = E ∪ { (v,z) | v∈V } => z este adiacent la toate celelalte noduri din V => z trebuie să aibă o culoare pe care nu o mai are niciun nod din V (ex: galben) => toate celelalte noduri din V, adică toate nodurile din V, sunt colorate cu numai 3 culori astfel încât ∄ două noduri adiacente în V colorate la fel => C \ { (z, galben) } colorare pentru 3-colorare

Set Partition $\equiv_p$ Subset Sum

Set Partition: Dându-se o mulțime X de întregi, se pot partiționa aceștia în două mulțimi A și X-A de sume egale ($\sum_{x\in A} x = \sum_{x\in X-A} x$)?
Ex: Pt X = {20, 23, 25, 45, 27, 40}, se pot forma mulțimile A = {20, 25, 45}, X-A = {23, 27, 40}.

Subset Sum: Dându-se o mulțime X de întregi și un întreg q, există o submulțime $Y \subseteq X$ a.î.. suma elementelor din Y să fie q ($(\sum_{x\in Y} x = q)$?
Ex: Pt X = {20, 23, 25, 45, 27, 40} și q = 63, există Y = {23, 40}.

Pentru a demonstra că cele două probleme sunt echivalente polinomial, demonstrăm pe rând că fiecare din cele două probleme se reduce la cealaltă.

Set Partition $\leq_p$ Subset Sum

[Pentru a avea soluție pentru Set Partition fix când avem soluție pentru Subset Sum, este suficient să alegem numărul q ca fiind suma unei partiții, adică (suma elementelor din X)/2]

Transformarea în timp polinomial a datelor de intrare:

Set Partition	**Subset Sum**
X	X = X
	q = $\sum_{x \in X} x$ / 2

=> ((A, X-A) partiție pt Set Partition => A și X-A soluții pt Subset Sum)

- (A, X-A) partiție pt Set Partition => $\sum_{x \in A} x = \sum_{x \in X-A} x = \sum_{x \in X} x / 2 = q$ => A și X-A soluții pt Subset Sum

<= (A soluție pt Subset Sum => (A, X-A) partiție pt Set Partition)

- A soluție pt Subset Sum => $\sum_{x \in A} x = q = \sum_{x \in X} x / 2 => \sum_{x \in X-A} x = \sum_{x \in X} x - \sum_{x \in X} x / 2 = \sum_{x \in X} x / 2 = q = \sum_{x \in A} x$ => (A, X-A) partiție pt Set Partition

Subset Sum $\leq_p$ Set Partition

[De data asta trebuie să construim o instanță convenabilă pt Set Partition dintr-o instanță oarecare pentru Subset Sum. Ideea este să forțăm partițiile să aibă fiecare suma q. Obținem acest efect adăugând în X elementul 2q – s, unde s = $\sum_{x \in X} x$, suma tuturor elementelor din instanța pentru Subset Sum]

Transformarea în timp polinomial a datelor de intrare:

Subset Sum	**Set Partition**
X (și fie s = $\sum_{x \in X} x$)	X = X ∪ {2q – s}
q	

=> (A soluție pt Subset Sum => (A, X-A) partiție pt Set Partition)

- A soluție pt Subset Sum => $\sum_{x \in A} x = q => \sum_{x \in X-A} x = \sum_{x \in X} x - \sum_{x \in A} x = \sum_{x \in X} x + 2q – s - \sum_{x \in A} x = s + 2q – s – q = q$ => (A, X-A) partiție pt Set Partition
- $\sum_{x \in A} x = \sum_{x \in X-A} x = \sum_{x \in X} x / 2 = q$ => A și X-A soluții pt Subset Sum

<= ((A, X-A) partiție pt Set Partition => aceea dintre A și X-A care nu îl conține pe 2q – s este soluție pt Subset Sum)

- (A, X-A) partiție pt Set Partition => $\sum_{x \in A} x = \sum_{x \in X-A} x = \sum_{x \in X} x / 2 = (s+2q-s)/2 = q$ => aceea dintre A și X-A care nu îl conține pe 2q – s este soluție pt Subset Sum (întrucât conține numai elemente din X, care au suma q)

Exercițiu: Să se arate că Resource Reservation Problem este o problemă NP-completă.

Resource Reservation Problem: Dându-se o mulțime P de procese, o mulțime R de resurse, o mulțime S = {S_1, S_2 ... S_n} cu S_i = mulțimea de resurse solicitate de procesul i, și un număr natural k, se pot aloca resurse proceselor astfel încât cel putin k procese să fie active simultan? (O resursă poate fi solicitată de oricâte procese, dar la un anumit moment poate fi folosită de un singur proces.)

Pt a arăta că RRP∈NP-hard, arătăm că **Independent Set $\leq_p$ RRP**.

Transformarea în timp polinomial a datelor de intrare:

Independent Set	**RRP**
(V,E)	P = V
k	R = E
	S_v = { e∈E | ∃ u∈V a.i. e=(u,v) }
	k = k

=> (S independent set => S soluție pt RRP)

- Fie S independent set și (u,v) o muchie arbitrară din G
- S independent set => u∉S sau v∉S
- Din definiția lui S_i: (u,v)∈S_u, (u,v)∈S_v și (u,v)∉S_w, pt w∉{u,v} => (u,v)∉S_i∩S_j, ∀i,j∈S cu i≠j
- Cum (u,v) a fost aleasă arbitrar => S este soluție pentru RRP

<= (S soluție pt RRP => S independent set)

- Fie S soluție pt RRP și (u,v) o resursă arbitrară din R (deci din E)
- Toate procesele din S pot fi active simultan => (u,v)∉S_i∩S_j, ∀i,j∈S cu i≠j
- => nu există 2 noduri distincte în S care să fie conectate prin muchia (u,v)
- Cum (u,v) a fost aleasă arbitrar => S este independent set

Pt a arăta ca RRP∈NP, trebuie să alegem una din următoarele două strategii:

- Scriem un algoritm nedeterminist polinomial care rezolvă RRP
- Arătăm că o soluție pt RRP poate fi verificată în timp polinomial

Alegem prima strategie:

```
RRP-NP(P,R,S,k){
       for i=1 to size(P) chosen[i] = false
       for i=1 to size(R) busy[i] = false
       for i=1 to k
              p = choice(1..size(P))
              if chosen[p] fail
              chosen[p] = true
              forall j in Sp
                     if busy[j] fail
                     busy[j] = true
       success
}
```

Am arătat că RRP este NP-hard și că RRP este NP => RRP este NP-completă.

Mai mulți algoritmi nedeterminiști

```
SET-COVER(U,S,k){
       for i=1 to size(U) covered[U[i]] = false
       for i=1 to size(S) chosen[i] = false
       for i=1 to k
              j = choice(1..size(S))
              if chosen[j] fail
              chosen[j] = true
              forall x in Sj
```

```
                    covered[x] = true
        for i=1 to size(U)
              if !covered[U[i]] fail
        success
}

SET-PARTITION(X){ // presupunem ca cele două partitii trebuie sa fie nevide
        s = 0 // suma tuturor numerelor din X
        for i=1 to size(X)
              chosen[i] = false
              s += X[i]
        q = 0 // suma unei partitii
        for i=1 to size(X)/2 // gasim partitia mai „scurta”
              j = choice(1..size(X))
              if chosen[j] fail
              chosen[j] = true
              q += X[j]
              if 2*q == s success
        fail
}
```